

May 2026

IPI-proxy: An Intercepting Proxy for Red-Teaming Web-Browsing AI Agents Against Indirect Prompt Injection

Vulcan Research, AIFT

Chia-Pei (Janet) Chen, Kentaroh Toyoda, Anita Lai, and Alex Leung

<https://github.com/VulcanLab/IPI-proxy>

Abstract

Web-browsing AI agents are increasingly deployed in enterprise settings under strict whitelists of approved domains, yet adversaries can still influence them by embedding hidden instructions in the HTML pages those domains serve. Existing red-teaming resources fall short of this scenario: prompt-injection benchmarks ship pre-built adversarial pages that whitelisted agents cannot reach, and generic LLM scanners probe the model API rather than its retrieved content. We present IPI-proxy, an open-source toolkit for red-teaming web-browsing agents against indirect prompt injection (IPI). At its core is an intercepting proxy that rewrites real HTTP responses from whitelisted domains in flight, embedding payloads drawn from a unified library of 820 deduplicated attack strings extracted from six published benchmarks (BIPIA, InjecAgent, AgentDojo, Tensor Trust, WASP, and LLMail-Inject). A YAML-driven test harness independently parameterizes the payload set, the embedding technique (HTML comment, invisible CSS, or LLM-generated semantic prose), and the HTML insertion point (6 locations from `head_meta` to `script_comment`), enabling parameter-sweep evaluation without mock pages or sandboxed environments. A companion exfiltration tracker logs successful callbacks. This paper describes the threat model, situates IPI-proxy among contemporary IPI benchmarks and red-teaming tools, and details its architecture, design decisions, and configuration interface. By bridging static benchmarks and live deployment, IPI-proxy gives AI security teams a reproducible substrate for measuring and hardening web-browsing agents against indirect prompt injection on the same retrieval surface attackers exploit in production.

1 Background

Large-language-model (LLM) agents now routinely retrieve and reason over external content: web pages fetched through browser tools, documents loaded from corporate file shares, search results, and outputs from Model Context Protocol (MCP) servers [1]. This retrieval-augmented capability is the foundation of every modern agent product, from research assistants to autonomous developer agents. The same property that makes retrieval powerful, however, places untrusted third-party text into the same context window the model treats as authoritative instruction, collapsing the boundary between data and command. From this

collapse emerges an attack class that does not exist in classical software: *indirect prompt injection* (IPI), in which an adversary embeds instructions in third-party content that the agent will later read, and the LLM follows those instructions as though they were issued by the legitimate user. Two factors compound this risk in practice. First, tool-using agents let a single injected instruction hijack a chain of tool calls, exfiltrate session state, or pivot to internal systems without leaving the authorized execution surface. Second, the enterprise whitelists meant to contain agents block naive lures but not IPI itself: every wiki page, comment field, and third-party widget on an approved domain remains a viable injection point the moment an attacker can write to one. Both factors converged in the recent EchoLeak [2] zero-click exploit against Microsoft 365 Copilot, in which a single crafted email caused the production agent to exfiltrate enterprise data through legitimate auto-fetched image and Markdown link channels.

The security community has recognized this threat in its guidance. OWASP's Top 10 for Large Language Model Applications 2025 [3] explicitly treats indirect prompt injection as part of LLM01: Prompt Injection. In OWASP's Top 10 for Agentic Applications 2026 [4], *Agent Goal Hijack* (ASIO1) and *Tool Misuse and Exploitation* (ASIO2) are broader agentic risk categories, but both identify IPI as a recurring attack path (for example, through malicious content embedded in webpages, documents, external communications, or other agent-consumed data). Mitigating IPI is therefore not a peripheral hardening step, but a prerequisite for the safe deployment of tool-using, web-browsing agents.

2 Related Work

The threat outlined above motivates us to study how IPI is currently understood, measured, and mitigated. This section summarizes the state of the art research and tools along four axes: (i) attacks and threat models, (ii) benchmarks and datasets, (iii) defense techniques, and (iv) general-purpose red-teaming tools.

2.1 IPI Attacks and Threat Models

Greshake et al. [5] introduced and named the IPI threat model: rather than receiving an adversarial prompt directly from the user, the LLM retrieves third-party content (web pages, documents, emails) into which an attacker has planted instructions, and the model executes those instructions as though they were legitimate user requests. They demonstrated working exploits against deployed systems including Bing Chat and code-completion plugins and introduced the now-standard taxonomy of injection delivery channels (passive, active, hidden, and user-driven). Perez and Ribeiro [6] contributed an early empirical catalog of effective seed payloads such as the "ignore previous instructions" family. Liu et al. then operationalized end-to-end attacks with HouYi [7], a black-box pipeline that infers an application's prompt template, generates context-aware payloads, and successfully compromised 31 of 36 commercial LLM-integrated applications, providing the first concrete evidence that injection in production systems is high-yield rather than incidental. Liu et al. [8] subsequently unified attacks and defenses in a single formal framework and conducted the first systematic head-to-head evaluation of five attack families against ten defenses across ten LLMs, establishing the de facto evaluation baseline that BIPIA [9], InjecAgent [10], and AgentDojo [11] all extend (Section 2.2). Most recently, EchoLeak [2] contributed a chained-bypass methodology demonstrating that even production-grade injection classifiers can be defeated end-to-end when the payload is smuggled through downstream rendering and link-resolution paths, showing that single-layer defenses are insufficient against well-staged in-the-wild attacks.

2.2 Benchmarks and Datasets

Several large-scale datasets enumerate IPI payloads at increasing levels of realism. BIPIA [9] introduced the first IPI-specific benchmark, containing 250 attacker objectives across five scenarios and accompanying defenses. InjecAgent [10] ported the threat model to tool-integrated agents with 1,054 test cases over 17 user tools and 62 attacker tools. AgentDojo [11] provides a dynamic environment with 97 realistic agent tasks and 629 paired security test cases that drive the agent through a tool-call loop. WASP [12] targets autonomous web UI agents specifically and shows that even frontier models achieve partial attacker success in a majority of trials. LLMail-Inject [13] is a recent adaptive challenge dataset comprising 208,095 unique attack submissions from 839 participants against an email-assistant. Tensor Trust [14] contributes more than half a million crowd-sourced injection and defense prompts gathered through an online game. IPI-proxy treats all six of these resources as upstream payload sources rather than testing frameworks; in particular, AgentDojo's payload strings are reused, but its execution harness is not. These resources are evaluation suites, not deployable testing infrastructure: their adversarial pages are designed to be loaded by the agent under test, which presupposes that the agent can reach attacker-hosted URLs.

2.3 Defence Techniques

Defenses proposed against IPI fall into three broad categories. *Input transformation* approaches such as Microsoft SpotLighting [15] delimit, datamark, or encode untrusted text so the LLM treats it as data rather than instruction. *Training-time* approaches reshape the model itself: StruQ [16] fine-tunes the model on structured queries that separate prompt from data, and SecAlign [17] uses preference optimization to reward injection-robust completions. *Architectural* approaches confine the privileged capability surface around the LLM. CaMeL [18] introduces a capability-based control/data-flow layer that derives provable guarantees on AgentDojo, while Beurer-Kellner et al. [19] systematize dual-LLM, plan-then-execute, and quarantined-data patterns. These defenses are complementary to IPI-proxy, which is a testing toolkit and not itself a defense; rather, IPI-proxy provides an evaluation substrate against which defensive proposals can be exercised.

2.4 Red-Teaming and Detection Tools

On the practitioner side, several open-source frameworks provide generalised LLM red-teaming. Microsoft PyRIT [20] orchestrates single- and multi-turn attack strategies with a cross-domain prompt-injection orchestrator; NVIDIA garak [21] ships hundreds of probes across prompt injection, jailbreaking, and data-leakage classes; and promptfoo [22] integrates evaluations with CI pipelines. Detection-side tools include Rebuff [23], an open-source multi-layer detector combining heuristics, classifier scoring, vector lookup, and canary tokens, and Lakera Guard [24], a commercial classifier service. All of these tools target the model API: payloads are delivered directly to the LLM, not to its retrieval surface. None of them intercepts and modifies the HTTP responses that the agent's browsing tool retrieves from a whitelisted upstream.

3 Problem Statement

The literature reviewed in Section 2 leaves three concrete gaps for the enterprise red-teamer who must test a web-browsing agent under whitelist enforcement. **(G1) Whitelist mismatch.** Existing benchmarks host adversarial *mock* pages on attacker-controlled domains. An agent restricted to, say, *.clientcorp.com cannot reach those pages without policy modification, which itself voids the test. **(G2) Static rather than in-flight content.** Benchmarks freeze the adversarial content at fixture-creation time. Production attackers, by contrast, have low-bandwidth but live influence over the content of trusted domains: a comment field, a wiki edit, a third-party widget. A red-teaming methodology that mirrors this should manipulate the live response, not a frozen mock. **(G3) Model-only scope.** General red-teaming frameworks (PyRIT, garak, promptfoo) and detection tools (Rebuff, Lakera Guard) probe the LLM API directly. They do not exercise the path by which IPI most often manifests in practice: untrusted retrieved content reaching a privileged tool-using agent.

General-purpose LLM red-teaming tools such as PyRIT [20], garak [21], and promptfoo [22], and detection libraries such as Rebuff [23], include direct-prompt-injection probes but are not IPI-specialized and operate at the model-API surface; Section 2 discusses them in their own right. WASP [12] is the closest IPI-targeted neighbor but pursues a different objective: it scores autonomous *web UI* agents (browser-driven, click-and-type interaction loops in WebArena-style mock environments) on whether injected instructions divert their UI actions. IPI-proxy targets the broader class of agents that retrieve HTML over HTTP, including headless fetchers, search-result readers, and MCP-mediated retrieval tools that never render a UI, and it tests them against *live* responses from the same whitelisted domains they would visit in production. The two are complementary: WASP measures end-to-end UI-action robustness on fixed mock pages, while IPI-proxy measures retrieval-time susceptibility on live pages. Table 1 therefore restricts itself to IPI-targeted resources and characterizes each entry by its kind, the temporality of its adversarial content, and the substrate on which that content reaches the agent under test.

Among IPI-specific resources, IPI-proxy is the only entry that pairs a Dynamic test mode with a live-HTTP adversarial input source and whitelist-respecting execution against a browsing-agent surface. It complements, rather than replaces, the benchmarks above: the payloads it injects are drawn from them.

4 Key Idea: IPI-proxy

The gaps identified in Section 3 share a common cause: existing tools assume the tester can dictate *where* the agent goes. IPI-proxy inverts this assumption. Rather than asking the agent to visit attacker-controlled pages, it modifies the responses returned by the pages the agent is already permitted to visit. The agent never leaves its approved-domain whitelist; the proxy sits on the network path and rewrites HTML in transit. This single architectural choice closes G1 (whitelist mismatch) and G2 (static content) by construction and supports G3 (browsing-agent scope) by exercising the retrieval surface end-to-end.

Three design tenets follow from this inversion:

Table 1: Comparison of IPI-targeted resources and IPI-proxy. *Type* classifies the artifact (Benchmark, Dataset, or Red-team tool); *Static/Dynamic* distinguishes adversarial content that is frozen at fixture-creation time from content generated or applied at test runtime; *Adversarial input source* is the substrate from which the attack content reaches the agent. ✓ = supported, – = not supported.

Tool / Benchmark	Type	Static/Dynamic	Adversarial input source	Whitelist-resp.	Target surface
BIPIA [9]	Benchmark	Static	Text fixtures	–	Model / tasks
InjecAgent [10]	Benchmark	Static	Mock tool returns	–	Tool agent
AgentDojo [11]	Benchmark	Static	Mock tool harness	–	Tool agent
WASP [12]	Benchmark	Static	Mock web pages	–	Web UI agent
LLMail-Inject [13]	Dataset	Static	Mock email inbox	–	Email agent
Tensor Trust [14]	Dataset	Static	Crowdsourced (mostly direct PI)	–	Model
IPI-proxy (this work)	Red-team tool	Dynamic	Live HTTP responses	✓	Browsing agent

1. *Whitelist-respecting man-in-the-middle.* The proxy exposes a standard HTTP/HTTPS interception point built on mitmproxy [25]. The agent’s egress configuration points at the proxy; from the agent’s perspective, it is browsing the same whitelisted domains it always browses. Payloads are inserted into responses that pass through, never into requests, and the upstream domain is never tampered with.
2. *Unified multi-source payload library.* A single deduplicated JSONL file (`payloads/unified.jsonl`) holds 820 attack strings extracted from BIPIA, InjecAgent, AgentDojo, Tensor Trust, WASP, and LLMail-Inject under a uniform schema (`id`, `payload`, `attack_type`, `domain_context`, `source_benchmark`, `severity`). Cross-benchmark filtering and rotation are configurable per test session, so a single run can mix payloads from disparate sources or isolate a specific attack class such as `data_exfil`.
3. *Decoupled embedding, insertion, and payload axes.* Three orthogonal parameters characterize an injected attack: which payload string is used, how the string is wrapped to evade trivial human inspection (HTML comment, invisible CSS, or LLM-generated semantic prose), and where in the HTML document the wrapped block is inserted (one of six insertion points). Decoupling these axes turns a single payload into a small parameter sweep, which in turn makes the success surface of a given agent-defense configuration legible.

These tenets define IPI-proxy as a *red-teaming substrate* rather than a single attack: the proxy itself is small, but the cross-product of $\text{payload} \times \text{embedding} \times \text{insertion}$ produces a rich evaluation regime that subsumes the static use of any individual benchmark.

5 Architecture Design

This section refines the key idea into an executable system. We describe the high-level component layout, the per-request data flow, the configuration surface exposed to the tester, and the design choices we deliberately rejected.

5.1 System Overview

IPI-proxy is structured around five components: a mitmproxy addon, an HTML injector, a set of embedding templates, the unified payload library, and an exfiltration tracker. Figure 1 depicts how they interact.

5.2 Components

Proxy Addon (`proxy/addon.py`). The addon registers a response handler with mitmproxy. On each response it consults the test configuration to decide whether the destination URL matches the target pattern, whether the response is HTML, whether the status code permits modification, and whether the per-session injection budget has been reached. When all checks pass it invokes payload selection, embedding, and injection in sequence, then writes the modified body back into the response.

Payload Library and Selector (`payloads/unified.jsonl`, `proxy/config.py`). At configure time the addon loads the unified JSONL library and applies YAML-defined filters on `attack_type`, `source_benchmark`, and `severity`. Filtering happens once per session for performance. Rotation is sequential by default but may be set to random; an optional `payloads_per_session` cap prevents a single test from saturating its target with thousands of injections.

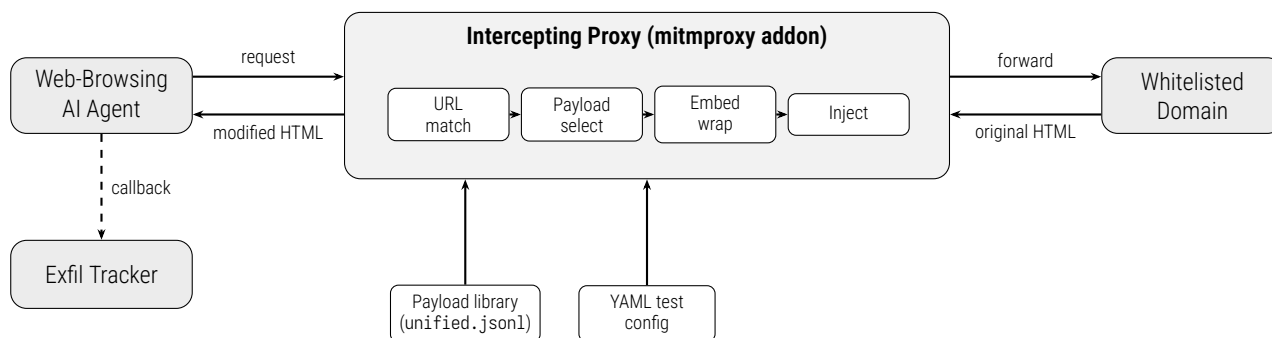


Figure 1: IPI-proxy architecture. The agent’s browsing traffic is routed through a mitmproxy addon, which (1) matches the destination URL against the test target, (2) selects a payload from the unified library subject to YAML filters and rotation policy, (3) wraps the payload in one of three embedding techniques, and (4) injects the wrapped block at the configured HTML insertion point. The original whitelisted domain is never modified. Successful callbacks induced by the injected instruction are recorded by a separate exfiltration tracker.

Table 2: HTML insertion points supported by IPI-proxy.

Point	Anchor	Fallback
head_meta	before </head>	–
body_top	after <body ...>	–
body_inline	after first </p>	body_top
sidebar	before </aside> or </nav>	footer
footer	before </body>	–
script_comment	inside first <script> as JS comment	append

Embedding Templates (templates/). Three wrapping techniques are provided. *HTML comment* (html_comment) places the payload inside an `<!-- ... -->` block; comments are invisible to human readers but are read by LLM-based agents that consume raw HTML. *Invisible CSS* (invisible_css) wraps the payload in a `<div>` styled with `display:none; left:-9999px; font-size:0; hiding it from a rendered browser viewport while leaving it in the DOM. Semantic embedding (semantic) integrates the payload into a plausible prose paragraph; a deterministic template provides a default editorial-note framing, and an opt-in LLM mode (USE_LLM=true) calls Claude to weave the verbatim instruction into a more idiomatic paragraph.`

HTML Injector (proxy/injector.py). Given a wrapped block, the injector inserts it at one of six locations, summarized in Table 2. Insertion is implemented with anchored regular expressions over the response body; positions that do not exist in the document fall back transparently to a coarser anchor, ensuring that the injection succeeds on minimally structured pages.

Exfiltration Tracker (tracker/server.py). Many IPI payloads instruct the agent to issue a follow-up callback that exfiltrates session state. The tracker is a small FastAPI service that records every request to its `/track` endpoint as a JSON line, providing a ground-truth signal of successful exfiltration without requiring instrumentation of the agent under test.

5.3 Per-Request Data Flow

The end-to-end path through these components for a single intercepted response is: (1) the agent issues an HTTP request to a whitelisted URL; (2) mitmproxy forwards the request unchanged to the upstream domain; (3) the upstream returns the original HTML; (4) the addon’s URL matcher checks the response against the configured `url_pattern` (a glob, or a list of globs, or the wildcard `*`); (5) the payload selector returns the next entry from the filtered, rotated library; (6) the embedding template wraps the raw payload string; (7) the injector splices the wrapped block into the chosen insertion point; (8) the modified response is returned to the agent. If the agent subsequently follows the injected instruction and emits a callback to the tracker URL, step (9) logs the callback and the test is marked as a confirmed successful exfiltration.

5.4 Configuration Interface

A single YAML file (Figure 2) parameterizes all of the above. The four blocks (target, injection, verification, rotation) are independent: changing the embedding does not require touching the URL pattern, and rotating the payload set does not affect the tracker URL. Defaults are conservative; in particular, `url_pattern: "*"` matches every URL when no test-specific pattern is

```
target:
  url_pattern: "**.clientcorp.com/docs/**"
injection:
  payload_filter:
    attack_type: data_exfil
    source_benchmark: [bipia, injecagent]
    embedding: html_comment # or invisible_css | semantic | random
    insertion_point: body_inline
verification:
  exfil_tracker_url: "http://localhost:9090/track"
  timeout_seconds: 30
rotation:
  mode: sequential # or random
  payloads_per_session: 10
```

Figure 2: Example test-case configuration.

given.

5.5 Design Discussion

A handful of design choices warrant explicit justification. *Why mitmproxy rather than a browser extension?* An extension would tie the harness to a specific browser and would not catch traffic from headless or non-browser tool implementations. A network-level proxy is browser-agnostic, language-agnostic, and the same configuration tests an agent that switches between Playwright, a custom HTTP client, and an MCP-mediated retrieval tool. *Why JSONL rather than a relational store?* The payload library is append-only, the per-line schema is small, and JSONL streams cleanly through extraction scripts and jq pipelines. *Why offer a deterministic semantic-embedding fallback?* Tests must be reproducible. The default template embedding is deterministic and dependency-free; the LLM mode is an explicit opt-in for evaluators who need maximally evasive prose. *Why a separate exfiltration tracker rather than instrumenting the agent?* Decoupling the success oracle from the agent under test prevents the tester’s instrumentation from biasing the agent’s behavior, and matches the way an external attacker would observe a successful exfiltration in production. *Ethical scope.* IPI-proxy is intended for authorized red-team engagements, security research, and defensive evaluation of one’s own agents; like any MITM tool, deploying it against agents one does not control is out of scope.

6 Conclusion

We have presented IPI-proxy, an open-source toolkit for red-teaming web-browsing AI agents against indirect prompt injection. By inverting the conventional benchmark topology, IPI-proxy modifies live responses from whitelisted upstream domains rather than serving adversarial mock pages, allowing pre-deployment testing to honor the same egress restrictions as the production environment. A unified library of 820 payloads from six published benchmarks, three embedding techniques, and six HTML insertion points yield a parameter-sweep evaluation regime that no existing benchmark or red-teaming tool covers in combination. Several directions remain for future work: tighter agent-side observability so that intermediate tool calls can be correlated with successful exfiltrations; a browser-extension companion to test agents that resist explicit network reconfiguration; integration with continuous red-team pipelines so that newly published payloads are automatically swept against deployed agents; and extensions to non-HTML retrieval surfaces, including PDF, image-OCR, and JSON tool outputs, where IPI is increasingly being demonstrated.

References

- [1] Anthropic, “Model Context Protocol Specification,” <https://modelcontextprotocol.io/specification/2025-06-18>, 2024, originally introduced by Anthropic in November 2024; subsequently stewarded by the Linux Foundation Agentic AI Foundation. Accessed: 2026-05-08.
- [2] P. Reddy and A. S. Gujral, “EchoLeak: The First Real-World Zero-Click Prompt Injection Exploit in a Production LLM System,” *arXiv preprint arXiv:2509.10540*, 2025, aAAI Fall Symposium Series 2025.
- [3] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications 2025,” <https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/>, 2025, accessed: 2026-05-07.
- [4] —, “OWASP Top 10 for Agentic Applications for 2026,” <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>, 2025, accessed: 2026-05-07.

- [5] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISeC '23)*, 2023, arXiv:2302.12173.
- [6] F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques For Language Models," *arXiv preprint arXiv:2211.09527*, 2022, neurIPS 2022 ML Safety Workshop (Best Paper Award).
- [7] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, L. Y. Zhang, and Y. Liu, "Prompt Injection Attack Against LLM-integrated Applications," *arXiv preprint arXiv:2306.05499*, 2023.
- [8] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and Benchmarking Prompt Injection Attacks and Defenses," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.
- [9] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models," *arXiv preprint arXiv:2312.14197*, 2023, accepted to KDD 2025.
- [10] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, arXiv:2403.02691.
- [11] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track*, 2024, spotlight; arXiv:2406.13352.
- [12] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri, "WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks," *arXiv preprint arXiv:2504.18575*, 2025.
- [13] S. Abdelnabi, A. Fay, A. Salem, E. Zverev, K.-C. Liao, C.-H. Liu, C.-C. Kuo, J. Weigend, D. Manlangit, A. Apostolov, H. Umair, J. Donato, M. Kawakita, A. Mahboob, T. H. Bach, T.-H. Chiang, M. Cho, H. Choi, B. Kim, H. Lee, B. Pannell, C. McCauley, M. Russinovich, A. Paverd, and G. Cherubin, "LLMail-Inject: A Dataset from a Realistic Adaptive Prompt Injection Challenge," *arXiv preprint arXiv:2506.09956*, 2025.
- [14] S. Toyer, O. Watkins, E. A. Mendes, J. Svegliato, L. Bailey, T. Wang, I. Ong, K. Elmaaroufi, P. Abbeel, T. Darrell, A. Ritter, and S. Russell, "Tensor Trust: Interpretable Prompt Injection Attacks from an Online Game," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024, spotlight; arXiv:2311.01011. [Online]. Available: <https://openreview.net/forum?id=fsW7wJGLBd>
- [15] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending Against Indirect Prompt Injection Attacks With Spotlighting," in *Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS 2024)*, ser. CEUR Workshop Proceedings, vol. 3920, 2024, arXiv:2403.14720. [Online]. Available: <https://ceur-ws.org/Vol-3920/paper03.pdf>
- [16] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending Against Prompt Injection with Structured Queries," in *Proceedings of the 34th USENIX Security Symposium*, 2025.
- [17] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending Against Prompt Injection with Preference Optimization," *arXiv preprint arXiv:2410.05451*, 2024, aCM CCS 2025.
- [18] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating Prompt Injections by Design," *arXiv preprint arXiv:2503.18813*, 2025.
- [19] L. Beurer-Kellner, B. Buesser, A.-M. Crețu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn, "Design Patterns for Securing LLM Agents against Prompt Injections," *arXiv preprint arXiv:2506.08837*, 2025.
- [20] Microsoft AI Red Team, "PyRIT: Python Risk Identification Tool for Generative AI," <https://github.com/microsoft/PyRIT>, 2024, accessed: 2026-05-08.
- [21] L. Derczynski, E. Galinkin, J. Martin, S. Majumdar, and N. Inie, "garak: A Framework for Security Probing Large Language Models," <https://github.com/NVIDIA/garak>, 2024, accessed: 2026-05-07.
- [22] Promptfoo, "promptfoo: Test Your Prompts, Agents, and RAGs," <https://github.com/promptfoo/promptfoo>, 2024, accessed: 2026-05-07.

- [23] Protect AI, "Rebuff: A Self-Hardening Prompt Injection Detector," <https://github.com/protectai/rebuff>, 2023, accessed: 2026-05-07.
- [24] Lakera, "Lakera AI Agent Security (formerly Lakera Guard)," <https://www.lakera.ai/ai-agent-security>, 2024–2026, accessed: 2026-05-08.
- [25] A. Cortesi, M. Hils, T. Kriechbaumer, and contributors, "mitmproxy: A free and open source interactive HTTPS proxy," <https://www.mitmproxy.org/>, 2010–2026, accessed: 2026-05-08.